

Structural Vibration Analyzer

Full Python Logger Documentation

Contents

1	Purpose of the Logger Script	5
2	Overall Logger Data Flow	6
3	Imported Python Packages	6
3.1	csv	6
3.1.1	Package Origin	6
3.1.2	Purpose	6
3.1.3	Important Function Used	6
3.2	struct	7
3.2.1	Package Origin	7
3.2.2	Purpose	7
3.2.3	Important Function Used	7
3.3	Path from pathlib	7
3.3.1	Package Origin	7
3.3.2	Purpose	7
3.3.3	Important Methods Used	8
3.4	datetime	8
3.4.1	Package Origin	8
3.4.2	Purpose	8
3.4.3	Important Function Used	8
3.5	serial	8
3.5.1	Package Origin	8
3.5.2	Purpose	8
3.5.3	Important Class Used	8
4	Logger Settings and Constants	9
4.1	Serial Port	9
4.1.1	Purpose	9
4.2	Baud Rate	9
4.2.1	Purpose	9
4.3	Raw Data Directory	9

4.3.1	Purpose	9
4.3.2	Function Origin	10
4.3.3	Inputs	10
4.3.4	Output	10
4.4	Number of Sensors	10
4.4.1	Purpose	10
4.5	Acceleration Scale	10
4.5.1	Purpose	10
4.6	Gyroscope Scale	10
4.6.1	Purpose	11
4.7	Synchronization Bytes	11
4.7.1	Purpose	11
4.8	Packet Type	11
4.8.1	Purpose	11
4.9	Payload Size	11
4.9.1	Purpose	12
4.10	Frame Size	12
4.10.1	Purpose	12
5	Output File Creation	13
5.1	Purpose	13
5.2	Functions Used	13
6	Function Breakdown	14
6.1	<code>calculate_checksum()</code>	14
6.1.1	Function Origin	14
6.1.2	Purpose	14
6.1.3	Input	14
6.1.4	Output	14
6.1.5	Mathematical Operation	14
6.1.6	How It Is Used	15
6.1.7	Library Functions Used	15
6.2	<code>read_until_binary_start()</code>	15
6.2.1	Function Origin	15
6.2.2	Purpose	15
6.2.3	Input	16
6.2.4	Output	16
6.2.5	Functions Used Inside	16
6.2.6	Why This Function Is Important	17
6.3	<code>read_exactly()</code>	17
6.3.1	Function Origin	17
6.3.2	Purpose	17
6.3.3	Inputs	17
6.3.4	Output	17
6.3.5	Functions Used Inside	18

6.3.6	Why This Function Is Needed	18
6.4	<code>read_frame()</code>	19
6.4.1	Function Origin	19
6.4.2	Purpose	19
6.4.3	Input	20
6.4.4	Output	20
6.4.5	Internal Custom Functions Used	20
6.4.6	Internal Library Functions Used	20
6.4.7	Step 1: Sync Byte Search	21
6.4.8	Step 2: Header Read	21
6.4.9	Step 3: Header Validation	21
6.4.10	Step 4: Payload Read	21
6.4.11	Step 5: Checksum Read	21
6.4.12	Step 6: Checksum Validation	22
6.5	<code>decode_payload()</code>	22
6.5.1	Function Origin	23
6.5.2	Purpose	23
6.5.3	Input	23
6.5.4	Output	23
6.5.5	Internal Library Functions Used	24
6.5.6	Step 1: Offset Initialization	24
6.5.7	Step 2: Decode Timestamp and Sample Index	24
6.5.8	Step 3: Decode Each Sensor	25
6.5.9	Step 4: Convert Acceleration to g	26
6.5.10	Step 5: Convert Gyroscope Values	26
6.5.11	Step 6: Calculate Resultant Acceleration	26
6.5.12	Step 7: Create Sensor Column Names	27
6.5.13	Step 8: Legacy Sensor-1 Columns	27
7	CSV Fieldname Generation	27
7.1	Purpose	28
7.2	Total Number of CSV Columns	29
8	Startup and Recording Loop	29
8.1	Startup Message	29
8.2	User Start Input	29
8.2.1	Function Origin	29
8.2.2	Purpose	29
8.3	Opening the Serial Port and CSV File	29
8.3.1	Functions Used	30
8.4	CSV Writer Creation	30
8.4.1	<code>csv.DictWriter()</code>	31
8.4.2	<code>writer.writeheader()</code>	31
8.5	Main Recording Loop	31
8.5.1	Purpose	31

8.5.2	Step 1: Read Frame	31
8.5.3	Step 2: Bad Frame Handling	32
8.5.4	Step 3: Decode Payload	32
8.5.5	Step 4: Write CSV Row	32
8.5.6	Step 5: Valid Frame Counter	32
8.6	Progress Message	32
8.6.1	Purpose	33
8.6.2	Time Conversion	33
9	Stopping the Logger	33
9.1	Function Origin	33
9.2	Purpose	33
9.3	Output	34
10	Binary Frame Decoding Summary	34
11	CSV Output Summary	34
12	Logger Reliability Features	35
12.1	Sync Byte Recovery	35
12.2	Packet Type Check	36
12.3	Payload Size Check	36
12.4	Checksum Check	36
12.5	Bad Frame Counter	36
13	Complete Logger Execution Flow	36
14	Summary	37

1 Purpose of the Logger Script

The purpose of the Python logger script, `logger_esp32.py`, is to receive the binary serial data sent by the ESP32, decode it into numerical sensor values, convert the raw readings into engineering units, and save the data into a CSV file.

The logger acts as the bridge between:

ESP32 Binary Serial Output → CSV Data File

The ESP32 sends compact binary packets because binary communication is faster and smaller than text-based communication. However, binary data is not directly readable by humans or convenient for analysis. Therefore, the logger converts the binary stream into a structured CSV file.

The logger performs the following main tasks:

1. Defines the serial port and baud rate.
2. Creates the raw-data output folder.
3. Creates a timestamped CSV output file.
4. Opens the serial port.
5. Waits for the ESP32 startup and calibration messages.
6. Detects the `START_BINARY` message from the ESP32.
7. Reads binary frames from the serial stream.
8. Finds synchronization bytes.
9. Checks packet type and payload size.
10. Verifies the checksum.
11. Decodes the binary payload.
12. Converts raw acceleration counts into g .
13. Converts raw gyroscope counts into degrees per second.
14. Calculates resultant acceleration.
15. Writes valid decoded data rows into a CSV file.
16. Skips corrupted or invalid frames.

2 Overall Logger Data Flow

The logger receives data in the following order:

ESP32 → USB Serial → PySerial → Binary Frame → Decoded Payload → CSV Row

The complete software pipeline is:

`Serial.read() → read_frame() → decode_payload() → csv.DictWriter.writerow()`

Each valid ESP32 binary frame becomes one row in the CSV file.

3 Imported Python Packages

The logger imports the following packages:

```
import csv
import struct
from pathlib import Path
from datetime import datetime

import serial
```

3.1 csv

3.1.1 Package Origin

csv is a standard Python library. It does not need to be installed separately.

3.1.2 Purpose

The csv package is used to write the decoded vibration data into a CSV file.

CSV means:

Comma-Separated Values

A CSV file stores tabular data using columns and rows.

3.1.3 Important Function Used

```
csv.DictWriter()
```

This function comes from the Python csv module.

It creates a writer object that writes dictionaries to a CSV file.

Each dictionary key becomes a column name.

Each dictionary value becomes the value in that column.

3.2 struct

3.2.1 Package Origin

`struct` is a standard Python library. It does not need to be installed separately.

3.2.2 Purpose

The `struct` package converts binary bytes into Python numerical values.

The ESP32 sends bytes, not readable text. Therefore, Python needs a way to interpret groups of bytes as integers.

3.2.3 Important Function Used

```
struct.unpack_from()
```

This function reads binary data from a byte buffer starting at a given offset.

For example:

```
struct.unpack_from("<II", payload, offset)
```

means:

- <: little-endian byte order
- I: unsigned 32-bit integer
- I: unsigned 32-bit integer

Therefore, "<II" decodes:

$t_{\mu s}$

and:

sample index

3.3 Path from pathlib

```
from pathlib import Path
```

3.3.1 Package Origin

`pathlib` is a standard Python library.

3.3.2 Purpose

`Path` is used to handle folder and file paths in a clean, readable way.

Instead of writing file paths as plain strings only, `Path` allows the script to create directories and combine paths more safely.

3.3.3 Important Methods Used

```
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

This creates the output folder if it does not already exist.

3.4 datetime

```
from datetime import datetime
```

3.4.1 Package Origin

`datetime` is a standard Python library.

3.4.2 Purpose

The `datetime` package is used to create a timestamped output filename.

This prevents new recordings from overwriting old recordings.

3.4.3 Important Function Used

```
datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
```

This creates a text timestamp such as:

2026-05-12_08-35-20

3.5 serial

```
import serial
```

3.5.1 Package Origin

`serial` comes from the external package `PySerial`.

It is not part of basic Python by default.

3.5.2 Purpose

`PySerial` allows Python to communicate with devices over serial ports.

In this project, it allows the computer to receive data from the ESP32 through USB.

3.5.3 Important Class Used

```
serial.Serial()
```

This opens a serial connection to the ESP32.

4 Logger Settings and Constants

4.1 Serial Port

```
PORT = "COM3"
```

4.1.1 Purpose

This defines the computer serial port connected to the ESP32.

For this script:

```
PORT = COM3
```

On Windows, ESP32 devices usually appear as COM ports.

If the ESP32 appears on a different port, this value must be changed.

4.2 Baud Rate

```
BAUD = 921600
```

4.2.1 Purpose

This defines the serial communication speed.

It must match the ESP32 firmware baud rate.

```
BAUD = 921600
```

If the logger baud rate and ESP32 baud rate do not match, the received data will be corrupted or unreadable.

4.3 Raw Data Directory

```
DATA_DIR = Path(r"F:\RLenovo99\Documents\Projects\Structural_Vibration_Analyzer\  
Data\raw")  
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

4.3.1 Purpose

This defines where the raw CSV files will be saved.

The call:

```
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

creates the folder if it does not exist.

4.3.2 Function Origin

`makedirs()` is a method from the `Path` object in Python's `pathlib` module.

4.3.3 Inputs

- `parents=True`: create missing parent folders if needed.
- `exist_ok=True`: do not raise an error if the folder already exists.

4.3.4 Output

No returned data is used.

The function creates the folder on the computer.

4.4 Number of Sensors

```
NUM_SENSORS = 3
```

4.4.1 Purpose

This tells the logger that each binary frame contains data from three sensors.

$$N_{\text{sensors}} = 3$$

This value must match the ESP32 firmware.

4.5 Acceleration Scale

```
ACCEL_SCALE = 4096.0
```

4.5.1 Purpose

This value converts raw accelerometer counts into acceleration in units of g .

The MPU6050 is configured for the $\pm 8g$ range.

At this range:

$$4096 \text{ counts} = 1g$$

Therefore:

$$a_g = \frac{a_{\text{raw}}}{4096}$$

4.6 Gyroscope Scale

```
GYRO_SCALE = 131.0
```

4.6.1 Purpose

This value converts raw gyroscope counts into angular velocity in degrees per second.

The MPU6050 is configured for:

$$\pm 250^{\circ}/s$$

At this range:

$$131 \text{ counts} = 1^{\circ}/s$$

Therefore:

$$\omega_{\text{deg}/s} = \frac{\omega_{\text{raw}}}{131}$$

4.7 Synchronization Bytes

```
SYNC = b"\xAA\x55"
```

4.7.1 Purpose

The synchronization bytes identify the beginning of each binary frame.

The ESP32 sends every frame starting with:

$$0xAA, 0x55$$

The logger searches for this exact two-byte sequence.

This allows the logger to recover if it starts reading in the middle of a frame.

4.8 Packet Type

```
PACKET_TYPE_DATA = 0x01
```

4.8.1 Purpose

This defines the expected packet type.

The ESP32 uses:

$$0x01$$

to indicate a data packet.

If the logger reads a packet with a different packet type, it rejects it.

4.9 Payload Size

```
PAYLOAD_SIZE = 4 + 4 + (NUM_SENSORS * 6 * 2)
```

4.9.1 Purpose

This defines the number of bytes in the payload portion of each frame.

The payload contains:

- 4 bytes for timestamp
- 4 bytes for sample index
- Sensor data

Each sensor sends six values:

$$a_x, a_y, a_z, g_x, g_y, g_z$$

Each value is a signed 16-bit integer:

2 bytes

For three sensors:

$$3 \times 6 \times 2 = 36 \text{ bytes}$$

Therefore:

$$\text{PAYLOAD_SIZE} = 4 + 4 + 36$$

$$\text{PAYLOAD_SIZE} = 44 \text{ bytes}$$

4.10 Frame Size

$\text{FRAME_SIZE} = 2 + 1 + 1 + \text{PAYLOAD_SIZE} + 1$

4.10.1 Purpose

This defines the full binary frame size.

The full frame contains:

- 2 sync bytes
- 1 packet type byte
- 1 payload size byte
- 44 payload bytes
- 1 checksum byte

Therefore:

$$\text{FRAME_SIZE} = 2 + 1 + 1 + 44 + 1$$

$$\text{FRAME_SIZE} = 49 \text{ bytes}$$

5 Output File Creation

```
timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
output_file = DATA_DIR / f"esp32_vibration_{timestamp}.csv"
```

5.1 Purpose

This creates a unique filename for each recording.

The filename format is:

esp32_vibration_YYYY-MM-DD_HH-MM-SS.csv

Example:

esp32_vibration_2026-05-12_08-45-30.csv

5.2 Functions Used

`datetime.now()` Origin:

`datetime`

Purpose:

Gets the current date and time.

`strftime()` Origin:

`datetime`

Purpose:

Formats the date and time into a string.

Path Join Operator / Origin:

`pathlib.Path`

Purpose:

Combines folder and filename into a complete file path.

6 Function Breakdown

6.1 calculate_checksum()

```
def calculate_checksum(frame_without_checksum: bytes) -> int:
    checksum = 0
    for byte in frame_without_checksum:
        checksum ^= byte
    return checksum
```

6.1.1 Function Origin

This is a custom function written specifically for this project.

It does not come from a Python library.

6.1.2 Purpose

This function calculates an XOR checksum for a binary frame.

The checksum is used to verify that the frame was received correctly.

6.1.3 Input

- `frame_without_checksum`: a `bytes` object containing all frame bytes except the final checksum byte.

6.1.4 Output

Returns an integer checksum value.

$$0 \leq \text{checksum} \leq 255$$

6.1.5 Mathematical Operation

The checksum is calculated using bitwise XOR:

$$C = b_0 \oplus b_1 \oplus b_2 \oplus \cdots \oplus b_n$$

where:

$$\oplus$$

represents XOR.

6.1.6 How It Is Used

The logger calculates the expected checksum after reading a frame.

It compares:

$$C_{\text{expected}}$$

against:

$$C_{\text{received}}$$

If they do not match, the frame is discarded.

6.1.7 Library Functions Used

No external library functions are used inside this function.

6.2 read_until_binary_start()

```
def read_until_binary_start(ser: serial.Serial) -> None:
    print("\nWaiting for ESP32 startup/calibration messages...\n")

    while True:
        line = ser.readline()

        if not line:
            continue

        text = line.decode("utf-8", errors="ignore").strip()

        if text:
            print(text)

        if text == "START_BINARY":
            print("\nBinary stream detected.")
            return
```

6.2.1 Function Origin

This is a custom function written specifically for this project.

6.2.2 Purpose

This function waits until the ESP32 finishes printing startup and calibration messages and sends:

START_BINARY

The logger does not begin reading binary frames until this exact message is detected.

6.2.3 Input

- **ser**: an active PySerial serial connection object.

6.2.4 Output

No value is returned.

The function exits only when:

```
text == "START_BINARY"
```

6.2.5 Functions Used Inside

ser.readline() Origin:

PySerial

Specifically:

```
serial.Serial.readline()
```

Purpose:

Reads one line of serial text ending in a newline character.

Output:

Returns raw bytes from the serial port.

decode() Origin:

Python **bytes** object method.

Purpose:

Converts raw bytes into a text string.

The script uses:

```
errors="ignore"
```

so invalid text characters are ignored instead of causing the logger to crash.

strip() Origin:

Python string method.

Purpose:

Removes leading and trailing spaces and newline characters.

print() Origin:

Python built-in function.

Purpose:

Displays ESP32 startup messages in the terminal.

6.2.6 Why This Function Is Important

Before binary data begins, the ESP32 prints readable text messages such as calibration status.

If the logger tried to read these text messages as binary frames, it would misinterpret the data.

This function prevents that by waiting for the clear transition marker:

START_BINARY

6.3 read_exactly()

```
def read_exactly(ser: serial.Serial, number_of_bytes: int) -> bytes:
    data = bytearray()

    while len(data) < number_of_bytes:
        chunk = ser.read(number_of_bytes - len(data))

        if not chunk:
            raise TimeoutError("Serial_timeout_while_reading_binary_packet.")

        data.extend(chunk)

    return bytes(data)
```

6.3.1 Function Origin

This is a custom function written specifically for this project.

6.3.2 Purpose

This function reads an exact number of bytes from the serial port.

Serial communication does not always return all requested bytes in one read operation. This function keeps reading until the required number of bytes has been received.

6.3.3 Inputs

- **ser**: active PySerial serial object.
- **number_of_bytes**: number of bytes that must be read.

6.3.4 Output

Returns a `bytes` object containing exactly:

`number_of_bytes`

`bytes.`

6.3.5 Functions Used Inside

`bytearray()` Origin:

Python built-in type.

Purpose:

Creates a mutable byte buffer.

A `bytearray` is used because data is collected in pieces.

`len()` Origin:

Python built-in function.

Purpose:

Checks how many bytes have already been collected.

`ser.read()` Origin:

`PySerial`

Purpose:

Reads bytes from the serial port.

`data.extend()` Origin:

Python `bytearray` method.

Purpose:

Adds the newly read bytes to the accumulated buffer.

`bytes()` Origin:

Python built-in type conversion.

Purpose:

Converts the mutable `bytearray` into an immutable `bytes` object.

`TimeoutError` Origin:

Python built-in exception type.

Purpose:

Raises an error if no bytes are received before the serial timeout.

6.3.6 Why This Function Is Needed

The logger needs exact byte counts because the binary frame format is fixed.

For example, after detecting sync bytes, the logger must read exactly:

2 header bytes

then exactly:

44 payload bytes

then exactly:

1 checksum byte

If the logger reads too few or too many bytes, the frame decoding will become misaligned.

6.4 read_frame()

```
def read_frame(ser: serial.Serial):
    previous = b""

    while True:
        current = ser.read(1)

        if not current:
            raise TimeoutError("Serial timeout while searching for sync bytes.")

        if previous + current == SYNC:
            break

        previous = current

    header_rest = read_exactly(ser, 2)
    packet_type = header_rest[0]
    payload_size = header_rest[1]

    if packet_type != PACKET_TYPE_DATA or payload_size != PAYLOAD_SIZE:
        return None

    payload = read_exactly(ser, PAYLOAD_SIZE)
    received_checksum = read_exactly(ser, 1)[0]

    frame_without_checksum = SYNC + header_rest + payload
    expected_checksum = calculate_checksum(frame_without_checksum)

    if received_checksum != expected_checksum:
        return None

    return payload
```

6.4.1 Function Origin

This is a custom function written specifically for this project.

6.4.2 Purpose

This function reads one complete binary frame from the ESP32.

It handles:

- Synchronization byte detection
- Header reading
- Packet type verification
- Payload size verification
- Payload reading
- Checksum verification

6.4.3 Input

- `ser`: active PySerial serial object.

6.4.4 Output

Returns:

- `payload`: if the frame is valid.
- `None`: if the frame is invalid or corrupted.

6.4.5 Internal Custom Functions Used

`read_exactly()` Origin:

Custom project function.

Purpose:

Reads a required number of bytes from the serial stream.

`calculate_checksum()` Origin:

Custom project function.

Purpose:

Calculates the expected checksum for the frame.

6.4.6 Internal Library Functions Used

`ser.read()` Origin:

PySerial

Purpose:

Reads raw bytes from the serial port.

6.4.7 Step 1: Sync Byte Search

The function searches byte-by-byte for:

`0xAA, 0x55`

These bytes are stored in:

```
SYNC = b"\xAA\x55"
```

This lets the logger recover if it starts reading in the middle of a frame.

6.4.8 Step 2: Header Read

After sync is found, the logger reads two bytes:

- Packet type
- Payload size

Header rest = 2 bytes

6.4.9 Step 3: Header Validation

The frame is accepted only if:

`packet_type = 0x01`

and:

`payload_size = 44`

If either value is wrong, the function returns:

None

6.4.10 Step 4: Payload Read

If the header is valid, the logger reads:

44 payload bytes

6.4.11 Step 5: Checksum Read

The logger then reads:

1 checksum byte

6.4.12 Step 6: Checksum Validation

The logger reconstructs the frame without the checksum:

sync bytes + header + payload

Then it calculates:

C_{expected}

If:

$C_{\text{received}} \neq C_{\text{expected}}$

the function returns:

None

If the checksum is valid, the function returns the payload.

6.5 decode_payload()

```
def decode_payload(payload: bytes):
    offset = 0

    t_us, sample_index = struct.unpack_from("<II", payload, offset)
    offset += 8

    row = {
        "t_us": t_us,
        "sample_index": sample_index,
    }

    legacy_values = {}

    for sensor_number in range(1, NUM_SENSORS + 1):
        raw_values = struct.unpack_from("<hhhhh", payload, offset)
        offset += 12

        ax_g = raw_values[0] / ACCEL_SCALE
        ay_g = raw_values[1] / ACCEL_SCALE
        az_g = raw_values[2] / ACCEL_SCALE
        gx_dps = raw_values[3] / GYRO_SCALE
        gy_dps = raw_values[4] / GYRO_SCALE
        gz_dps = raw_values[5] / GYRO_SCALE
        a_resultant_g = (ax_g**2 + ay_g**2 + az_g**2) ** 0.5

        prefix = f"s{sensor_number}"
```

```

row[f"{prefix}_ax_g"] = ax_g
row[f"{prefix}_ay_g"] = ay_g
row[f"{prefix}_az_g"] = az_g
row[f"{prefix}_gx_dps"] = gx_dps
row[f"{prefix}_gy_dps"] = gy_dps
row[f"{prefix}_gz_dps"] = gz_dps
row[f"{prefix}_a_resultant_g"] = a_resultant_g

if sensor_number == 1:
    legacy_values = {
        "ax_g": ax_g,
        "ay_g": ay_g,
        "az_g": az_g,
        "gx_dps": gx_dps,
        "gy_dps": gy_dps,
        "gz_dps": gz_dps,
        "a_resultant_g": a_resultant_g,
    }

row.update(legacy_values)
return row

```

6.5.1 Function Origin

This is a custom function written specifically for this project.

6.5.2 Purpose

This function decodes the 44-byte binary payload into a Python dictionary. The dictionary is later written as one CSV row.

6.5.3 Input

- payload: a valid 44-byte binary payload from the ESP32.

6.5.4 Output

Returns a Python dictionary named `row`.

The dictionary contains:

- Timestamp
- Sample index
- Acceleration values for sensors 1–3
- Gyroscope values for sensors 1–3

- Resultant acceleration for sensors 1–3
- Legacy single-sensor column names for sensor 1

6.5.5 Internal Library Functions Used

`struct.unpack_from()` Origin:

`struct`

Purpose:

Extracts typed numerical values from a binary byte buffer.

`range()` Origin:

Python built-in function.

Purpose:

Creates the sequence of sensor numbers:

1, 2, 3

`row.update()` Origin:

Python dictionary method.

Purpose:

Adds the legacy sensor-1 values into the main row dictionary.

6.5.6 Step 1: Offset Initialization

The function begins with:

`offset = 0`

The offset tracks where the next binary value begins inside the payload.

6.5.7 Step 2: Decode Timestamp and Sample Index

```
t_us, sample_index = struct.unpack_from("<II", payload, offset)
```

The format string:

`"<II"`

means:

- `<`: little-endian byte order
- `I`: unsigned 32-bit integer
- `I`: unsigned 32-bit integer

This reads:

$$t_{\mu s}$$

and:

sample index

The offset then advances by:

8 bytes

because:

$$4 + 4 = 8$$

6.5.8 Step 3: Decode Each Sensor

For each sensor, the code reads:

```
raw_values = struct.unpack_from("<hhhhh", payload, offset)
```

The format string:

"<hhhhh"

means:

- <: little-endian byte order
- h: signed 16-bit integer
- six values total

The six values are:

$$a_x, a_y, a_z, g_x, g_y, g_z$$

Each value uses:

2 bytes

Therefore, each sensor uses:

$$6 \times 2 = 12 \text{ bytes}$$

The offset advances by:

12 bytes

after each sensor.

6.5.9 Step 4: Convert Acceleration to g

The raw acceleration values are converted using:

$$a_x[g] = \frac{a_{x,\text{raw}}}{4096}$$

$$a_y[g] = \frac{a_{y,\text{raw}}}{4096}$$

$$a_z[g] = \frac{a_{z,\text{raw}}}{4096}$$

This is done in the code as:

```
ax_g = raw_values[0] / ACCEL_SCALE  
ay_g = raw_values[1] / ACCEL_SCALE  
az_g = raw_values[2] / ACCEL_SCALE
```

6.5.10 Step 5: Convert Gyroscope Values

The gyroscope values are converted into degrees per second:

$$g_x[^\circ/s] = \frac{g_{x,\text{raw}}}{131}$$

$$g_y[^\circ/s] = \frac{g_{y,\text{raw}}}{131}$$

$$g_z[^\circ/s] = \frac{g_{z,\text{raw}}}{131}$$

This is done as:

```
gx_dps = raw_values[3] / GYRO_SCALE  
gy_dps = raw_values[4] / GYRO_SCALE  
gz_dps = raw_values[5] / GYRO_SCALE
```

6.5.11 Step 6: Calculate Resultant Acceleration

The resultant acceleration is calculated as:

$$a_{\text{resultant}} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

In code:

```
a_resultant_g = (ax_g**2 + ay_g**2 + az_g**2) ** 0.5
```

This value represents the magnitude of the 3D acceleration vector.

6.5.12 Step 7: Create Sensor Column Names

The prefix:

`s1, s2, s3`

is created using:

```
prefix = f"s{sensor_number}"
```

This allows columns such as:

`s1_ax_g`

`s2_ay_g`

`s3_az_g`

6.5.13 Step 8: Legacy Sensor-1 Columns

For sensor 1, the script also creates older column names:

- `ax_g`
- `ay_g`
- `az_g`
- `gx_dps`
- `gy_dps`
- `gz_dps`
- `a_resultant_g`

These allow older single-sensor analysis scripts or habits to still work.

7 CSV Fieldname Generation

```
fieldnames = ["t_us", "sample_index"]

for sensor_number in range(1, NUM_SENSORS + 1):
    prefix = f"s{sensor_number}"
    fieldnames += [
        f"{prefix}_ax_g",
        f"{prefix}_ay_g",
        f"{prefix}_az_g",
```

```

        f"{prefix}_gx_dps",
        f"{prefix}_gy_dps",
        f"{prefix}_gz_dps",
        f"{prefix}_a_resultant_g",
    ]

fieldnames += [
    "ax_g",
    "ay_g",
    "az_g",
    "gx_dps",
    "gy_dps",
    "gz_dps",
    "a_resultant_g",
]
```

7.1 Purpose

This section defines the CSV column order.

The first columns are:

t_us

and:

sample_index

Then the script adds seven columns per sensor:

- ax_g
- ay_g
- az_g
- gx_dps
- gy_dps
- gz_dps
- a_resultant_g

For three sensors:

$$3 \times 7 = 21$$

multi-sensor columns are created.

Then seven legacy sensor-1 columns are added.

7.2 Total Number of CSV Columns

The total number of columns is:

$$2 + 21 + 7 = 30$$

8 Startup and Recording Loop

8.1 Startup Message

The logger prints:

```
print("\n=====")
print("_ESP32_3-MPU6050_BINARY_VIBRATION_LOGGER")
print("=====")
```

This displays the logger title.

It also prints:

- COM port
- Baud rate
- Output file path

8.2 User Start Input

```
input("\nPress ENTER to START recording...")
```

8.2.1 Function Origin

`input()` is a Python built-in function.

8.2.2 Purpose

It waits until the user presses ENTER before opening the serial port and recording data.

This gives the user control over when the recording starts.

8.3 Opening the Serial Port and CSV File

```
with serial.Serial(PORT, BAUD, timeout=2) as ser, open(output_file, "w", newline="")
    as f:
```

8.3.1 Functions Used

`serial.Serial()` Origin:

PySerial

Purpose:

Opens the serial connection to the ESP32.

Inputs:

- `PORT`
- `BAUD`
- `timeout=2`

Output:

Returns a serial object named:

`ser`

`open()` Origin:

Python built-in function.

Purpose:

Opens the output CSV file.

Inputs:

- File path
- Mode "`w`" for writing
- `newline=""` to prevent extra blank lines in CSV output

Output:

Returns a file object named:

`f`

with Context Manager Origin:

Python language feature.

Purpose:

Automatically closes the serial port and file when the block ends.

8.4 CSV Writer Creation

```
writer = csv.DictWriter(f, fieldnames=fieldnames)
writer.writeheader()
```

8.4.1 csv.DictWriter()

Origin:

CSV

Purpose:

Creates a CSV writer that writes dictionaries.

Input:

- File object
- Fieldnames list

Output:

A writer object.

8.4.2 writer.writeheader()

Origin:

Method of csv.DictWriter.

Purpose:

Writes the first row of the CSV file containing column names.

8.5 Main Recording Loop

```
while True:
    payload = read_frame(ser)

    if payload is None:
        bad_frames += 1
        continue

    row = decode_payload(payload)
    writer.writerow(row)
    valid_frames += 1
```

8.5.1 Purpose

This loop records data continuously until the user stops it with:

CTRL + C

8.5.2 Step 1: Read Frame

```
payload = read_frame(ser)
```

This attempts to read one valid binary frame.

8.5.3 Step 2: Bad Frame Handling

If the frame is invalid:

```
        payload is None

    then:

        bad_frames

    is increased by one.
    The loop then continues to the next frame.
```

8.5.4 Step 3: Decode Payload

If the frame is valid:

```
row = decode_payload(payload)
```

The binary payload becomes a dictionary.

8.5.5 Step 4: Write CSV Row

```
writer.writerow(row)
```

`writer.writerow()` Origin:

Method of `csv.DictWriter`.

Purpose:

Writes one decoded sensor sample to the CSV file.

8.5.6 Step 5: Valid Frame Counter

```
        valid_frames

    is increased by one.
```

8.6 Progress Message

```
if valid_frames % 250 == 0:
    print(
        f"Samples: {valid_frames}|"
        f"t={row['t_us']/1_000_000.0:.3f}|s|"
        f"bad_frames: {bad_frames}",
        end="\r",
    )
```


8.6.1 Purpose

Every 250 valid frames, the logger prints a status update.
It displays:

- Number of valid samples recorded
- Current time in seconds
- Number of bad frames skipped

8.6.2 Time Conversion

The timestamp is stored in microseconds:

$$t_{\mu s}$$

It is converted to seconds using:

$$t_s = \frac{t_{\mu s}}{1,000,000}$$

9 Stopping the Logger

```
except KeyboardInterrupt:
    print("\n\nRecording stopped by user.")
    print(f"Valid samples saved: {valid_frames}")
    print(f"Bad/corrupted frames skipped: {bad_frames}")
    print(f"\nData saved to: {output_file}")
    print("\nDone.")
```

9.1 Function Origin

KeyboardInterrupt is a Python built-in exception.

9.2 Purpose

When the user presses:

CTRL + C

Python raises a KeyboardInterrupt.

The script catches this interruption and exits cleanly.

9.3 Output

The logger prints:

- Number of valid samples saved
- Number of bad frames skipped
- Output file path

The CSV file remains saved because the `with` context manager closes it properly.

10 Binary Frame Decoding Summary

Each valid frame has the structure:

Field	Size	Meaning
Sync bytes	2 bytes	0xAA 0x55
Packet type	1 byte	Data packet marker 0x01
Payload size	1 byte	Expected value: 44
Timestamp	4 bytes	Microseconds since stream start
Sample index	4 bytes	Sample counter
Sensor 1 data	12 bytes	ax, ay, az, gx, gy, gz
Sensor 2 data	12 bytes	ax, ay, az, gx, gy, gz
Sensor 3 data	12 bytes	ax, ay, az, gx, gy, gz
Checksum	1 byte	XOR checksum

The logger only writes a row to the CSV if:

sync bytes are correct

packet type is correct

payload size is correct

checksum is correct

11 CSV Output Summary

Each valid binary frame becomes one CSV row.

The important columns include:

- `t_us`

- `sample_index`
- `s1_ax_g`
- `s1_ay_g`
- `s1_az_g`
- `s2_ax_g`
- `s2_ay_g`
- `s2_az_g`
- `s3_ax_g`
- `s3_ay_g`
- `s3_az_g`

The logger also saves gyroscope values:

- `s1_gx_dps`
- `s1_gy_dps`
- `s1_gz_dps`
- `s2_gx_dps`
- `s2_gy_dps`
- `s2_gz_dps`
- `s3_gx_dps`
- `s3_gy_dps`
- `s3_gz_dps`

12 Logger Reliability Features

12.1 Sync Byte Recovery

The logger searches for:

`0xAA, 0x55`

before reading a frame.

This allows the logger to recover from misalignment.

12.2 Packet Type Check

The logger verifies:

```
packet_type = 0x01
```

This prevents invalid packet types from being decoded.

12.3 Payload Size Check

The logger verifies:

```
payload_size = 44
```

This ensures the frame matches the expected three-sensor format.

12.4 Checksum Check

The logger recalculates the XOR checksum.

If the received checksum does not match, the frame is rejected.

12.5 Bad Frame Counter

The variable:

```
bad_frames
```

counts corrupted or invalid packets.

This helps evaluate communication reliability.

13 Complete Logger Execution Flow

The complete logger flow is:

1. Import required packages.
2. Define serial and binary-frame settings.
3. Create the raw data output folder.
4. Generate a timestamped output CSV filename.
5. Define helper functions.
6. Define CSV field names.
7. Print logger startup message.
8. Wait for user to press ENTER.

9. Open serial port.
10. Open output CSV file.
11. Create CSV writer.
12. Write CSV header.
13. Wait for ESP32 `START_BINARY` message.
14. Start continuous recording loop.
15. Search for frame sync bytes.
16. Read and validate frame header.
17. Read payload.
18. Read checksum.
19. Verify checksum.
20. Decode payload.
21. Convert raw counts to engineering units.
22. Write decoded row to CSV.
23. Print progress every 250 samples.
24. Continue until user presses `CTRL + C`.
25. Print final recording summary.

14 Summary

The Python logger is responsible for converting the ESP32 binary stream into a usable CSV file.

The most important custom functions are:

- `calculate_checksum()`
- `read_until_binary_start()`
- `read_exactly()`
- `read_frame()`
- `decode_payload()`

The most important imported functions and classes are:

- `serial.Serial()` from `PySerial`
- `ser.read()` from `PySerial`
- `ser.readline()` from `PySerial`
- `struct.unpack_from()` from the `struct` module
- `csv.DictWriter()` from the `csv` module
- `writer.writerow()` from the CSV writer object
- `Path.mkdir()` from `pathlib`
- `datetime.now()` from `datetime`

The logger makes the system reliable by checking:

- Synchronization bytes
- Packet type
- Payload size
- Checksum

Only valid frames are written to the CSV file.

The resulting CSV file is then used by the analyzer script to perform time-domain and frequency-domain vibration analysis.